

Loan Approval and Risk Assessment System

By Martin Waweru

1.0 Business Understanding

1.1 Introduction

PrimeTrust, a growing microfinance institution, faces challenges in managing a high volume of loan applications. Manual reviews are slow, subjective, and prone to error. The institution has hired our data team to develop a machine learning system that can automate and improve loan decision-making.

1.2 Problem Statement

PrimeTrust receives hundreds of loan applications each month from clients with different financial situations. The current manual process to approve loans is inefficient and inconsistent. Staff find it hard to assess:

- Who qualifies for a loan
- What amount to award
- What interest to charge
- Reasonable payment plan

This slows down the process and increases default risk. There's a need for an automated system that can make accurate, consistent, and fast decisions.

1.3 Stakeholder Questions

The following key questions from stakeholders will guide the development and evaluation of the loan approval and prediction system for Jirani Credit.

- Can we quickly and reliably determine if someone qualifies for a loan?
- If yes, how much loan should we offer them?
- What will be the interest rate charged?
- What will be their expected monthly repayment period (EMI)?
- Can we track this information in real time?
- Can we visualize loan approval trends, customer types, and risk levels?

PrimeTrust wants to adopt a ***risk-based interest rate approach***. This method allows the institution to assign interest rates based on each client's individual risk profile, rather than using fixed brackets. By doing so, PrimeTrust can better protect its portfolio from defaults, reward lower-risk clients with fairer rates, and build a more sustainable lending model as it grows.

1.4 Project Objectives

We will build a three-part solution:

- Classification model to predict loan approval (Approved/Rejected)
- Regression model to estimate the loan amount a qualified applicant can get
- Regression model to estimate the interest rate that will be charged
- EMI calculator to compute monthly repayment based on predicted amount and interest rate

Additional goals:

- Store user submissions (with name, email, and inputs) in database
- Build a Power BI dashboard for real-time tracking and reporting
- Create a web interface where staff can input new applications and get instant results

2.0 Data Understanding

2.1 Data Source

The dataset for this project was sourced from Kaggle. It contains customer demographic details, financial information, loan history, and approval status. This data will be used to train both classification and regression models for loan approval prediction and loan amount estimation.

[downloaded Dataset](#)

2.2 Data Description

The dataset contains historical loan application records. Each row represents one loan application, with information about the applicant, the loan details, and the approval outcome.

Column Name	Description
person_age	Age of the loan applicant (in years).
person_gender	Gender of the person.
person_education	Shows the education level of the client that is Highest education level e.g. masters, bachelors.
person_income	Annual income of the applicant (numeric, in local currency units).
person_home_ownership	Housing situation of the applicant: e.g. RENT, MORTGAGE, OWN, OTHER.
person_employment	Length of employment (in years). Zero often means unemployed or very new to a job.
loan_intent	Purpose of the loan, e.g. EDUCATION, PERSONAL, HOMEIMPROVEMENT, MEDICAL, VENTURE, etc.
loan_amnt	The amount of loan requested or issued (numeric).
loan_int_rate	Interest rate (%) applied to the loan. Sometimes missing (NaN) for unissued or rejected loans.
loan_status	Target variable (loan outcome): 1 means approved/paid (good loan), 0 means default/charge-off (bad loan).
credit_score	Shows the credit score of the client.
loan_percent_income	Ratio of loan amount to applicant's income ($\text{loan_amnt} \div \text{person_income}$).

Column Name	Description
come	Gives affordability measure.
cb_person_default_on_file	Whether applicant has ever defaulted before (from credit bureau records): Y = Yes, N = No.
cb_person_cred_hist_length	Length of applicant's credit history (in years). Indicates financial track record depth.

3.0 Data Preparation

```
# importing libraries
# Data Manipulation
import pandas as pd
import numpy as np
pd.options.display.float_format = '{:20.2f}'.format
pd.set_option('display.max_columns', 999)

# Data Visualizatiion
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

# Statistical analysis
from scipy import stats
from scipy.stats import chi2_contingency
from scipy.stats import f_oneway
from sklearn.preprocessing import StandardScaler
from statsmodels.tools.tools import add_constant
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant

# Modelling
from sklearn.ensemble import RandomForestClassifier,
RandomForestRegressor
from sklearn.linear_model import LogisticRegression
from xgboost import XGBClassifier, XGBRegressor
from sklearn.linear_model import LinearRegression
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE
from sklearn.pipeline import Pipeline
from sklearn.model_selection import RandomizedSearchCV
from sklearn.metrics import pairwise_distances_argmin_min

# metrics
from sklearn.metrics import classification_report, roc_auc_score
```

```

from sklearn.metrics import confusion_matrix
from sklearn.metrics import precision_recall_fscore_support
from sklearn.metrics import r2_score, mean_absolute_error,
mean_squared_error

# save
import joblib
import pickle

# filter feature warnings
import warnings
warnings.filterwarnings("ignore")

```

3.1 Data Overview

```

#load dataset
df1 = pd.read_csv(r"C:\Users\PC\Documents\After School\Project\
Project Main Box\Intelligent-Loan-Approval-System-and-Risk-Assessment-
System\Data\loan_data.csv")

# preview
df1.iloc[305:310]

```

	person_income \	person_age	person_gender	person_education
305	19169.00	21.00	female	Associate
306	118417.00	24.00	female	High School
307	133476.00	26.00	female	High School
308	19312.00	23.00	female	Associate
309	192351.00	25.00	male	High School

	person_emp_exp	person_home_ownership	loan_amnt \
305	2	MORTGAGE	1500.00
306	0	RENT	25000.00
307	0	RENT	25000.00
308	4	MORTGAGE	2800.00
309	1	RENT	30000.00

	loan_intent	loan_int_rate	loan_percent_income \
305	EDUCATION	10.59	0.08
306	HOMEIMPROVEMENT	12.61	0.21
307	PERSONAL	11.01	0.19
308	PERSONAL	8.49	0.14
309	PERSONAL	17.27	0.16

	cb_person_cred_hist_length	credit_score
previous_loan_defaults_on_file \		
305	2.00	537
Yes		
306	2.00	625
No		
307	2.00	641
Yes		
308	3.00	661
No		
309	4.00	577
No		

	loan_status
305	0
306	1
307	0
308	1
309	1

check the shape of the data

```
print(f"The dataset contains {df1.shape[0]} rows and {df1.shape[1]} columns")
```

The dataset contains 45000 rows and 14 columns

check statistical distribution

```
display(df1.describe())
display(df1.describe(include='O'))
```

	person_age	person_income
person_emp_exp \		
count	45000.00	45000.00
mean	27.76	80319.05
std	6.05	80422.50
min	20.00	8000.00
25%	24.00	47204.00
50%	26.00	67048.00
75%	30.00	95789.25
max	144.00	7200766.00

	loan_amnt	loan_int_rate
loan_percent_income \		

count	45000.00	45000.00	45000.00
mean	9583.16	11.01	0.14
std	6314.89	2.98	0.09
min	500.00	5.42	0.00
25%	5000.00	8.59	0.07
50%	8000.00	11.01	0.12
75%	12237.25	12.99	0.19
max	35000.00	20.00	0.66

cb_person_cred_hist_length		credit_score	
loan_status			
count	45000.00	45000.00	
45000.00			
mean	5.87	632.61	
0.22			
std	3.88	50.44	
0.42			
min	2.00	390.00	
0.00			
25%	3.00	601.00	
0.00			
50%	4.00	640.00	
0.00			
75%	8.00	670.00	
0.00			
max	30.00	850.00	
1.00			

person_gender		person_education	person_home_ownership
loan_intent \			
count	45000	45000	45000
45000			
unique	2	5	4
6			
top	male	Bachelor	RENT
EDUCATION			
freq	24841	13399	23443
9153			

previous_loan_defaults_on_file	
count	45000
unique	2

```
top          Yes
freq        22858
```

```
# check more info
```

```
df1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 45000 entries, 0 to 44999
```

```
Data columns (total 14 columns):
```

#	Column	Non-Null Count	Dtype
0	person_age	45000 non-null	float64
1	person_gender	45000 non-null	object
2	person_education	45000 non-null	object
3	person_income	45000 non-null	float64
4	person_emp_exp	45000 non-null	int64
5	person_home_ownership	45000 non-null	object
6	loan_amnt	45000 non-null	float64
7	loan_intent	45000 non-null	object
8	loan_int_rate	45000 non-null	float64
9	loan_percent_income	45000 non-null	float64
10	cb_person_cred_hist_length	45000 non-null	float64
11	credit_score	45000 non-null	int64
12	previous_loan_defaults_on_file	45000 non-null	object
13	loan_status	45000 non-null	int64

```
dtypes: float64(6), int64(3), object(5)
```

```
memory usage: 4.8+ MB
```

3.2 Exploratory Data Analysis

3.2.1 Data Cleaning Plan

if Any

- Drop Irrelevant Columns like Remove IDs, unnamed columns, or anything not useful for analysis.
- Fix Data Types that is, ensure each column has the appropriate data type (e.g. int, float, category, datetime).
- Handling Missing Data Fill or drop missing values based on relevance.
- Removing Duplicates if any exist.
- Detect and remove extreme values that could distort analysis that is remove outliers
- Clean and standardize text data Fix inconsistent labels (e.g., "Male" vs "male"), rename columns clearly, and remove extra spaces or typos.
- Perform Univariate Analysis i.e Understand the distribution of individual features (one at a time).
- Bivariate Analysis that is Explore relationships between two variables.
- Multivariate Analysis (if needed) Use heatmaps, pairplots, or pivot tables to analyze interactions among 3+ variables.

- Feature Insights that is Summarize the most useful features for modeling, and drop or flag irrelevant ones.

NB: for the missing values this is the criteria that will be used:

- If a column has 0% missing values, no cleaning is needed.
- If a column has less than 5% missing values, you can either drop the missing rows or fill them using mean/median/mode.
- If a column has more than 50% missing values, it may be better to drop the column entirely.

```
# Copy data to preserve the original data
```

```
df = df1.copy()
```

```
df.columns
```

```
Index(['person_age', 'person_gender', 'person_education',
      'person_income',
      'person_emp_exp', 'person_home_ownership', 'loan_amnt',
      'loan_intent',
      'loan_int_rate', 'loan_percent_income',
      'cb_person_cred_hist_length',
      'credit_score', 'previous_loan_defaults_on_file',
      'loan_status'],
      dtype='object')
```

```
# rename the columns to be readbale easily
```

```
df.rename(columns={
    'person_age': 'age',
    'person_gender': 'gender',
    'person_education': 'education',
    'person_income': 'income',
    'person_emp_exp': 'employment_experience',
    'person_home_ownership': 'home_ownership',
    'loan_amnt': 'loan_amount',
    'loan_intent': 'loan_purpose',
    'loan_int_rate': 'interest_rate',
    'loan_percent_income': 'dti_ratio',
    'cb_person_cred_hist_length': 'credit_history_length',
    'credit_score': 'credit_score',
    'previous_loan_defaults_on_file': 'default_history',
    'loan_status': 'loan_outcome'
}, inplace=True)
```

```
# preview
```

```
df.columns
```

```
Index(['age', 'gender', 'education', 'income',
      'employment_experience',
      'home_ownership', 'loan_amount', 'loan_purpose',
      'interest_rate',
```



```

        'dti_ratio', 'credit_history_length', 'credit_score',
        'default_history',
        'loan_outcome'],
        dtype='object')

```

Check current values BEFORE modifying

```
print("Education values before:", df['education'].unique())
```

```
Education values before: ['Master' 'High School' 'Bachelor'
                          'Associate' 'Doctorate']
```

```
df['education'] = df['education'].str.upper()
```

check the data types

```
df.dtypes
```

```

age                float64
gender             object
education          object
income            float64
employment_experience  int64
home_ownership     object
loan_amount        float64
loan_purpose         object
interest_rate      float64
dti_ratio          float64
credit_history_length float64
credit_score       int64
default_history    object
loan_outcome       int64
dtype: object

```

To make the dataset easier to work with, I adjusted some column data types to reduce memory use and prepare it for machine learning.

- **Categorical variables** – home_ownership, and loan_purpose were changed from object to category, since they contain a small set of repeating labels rather than continuous values.
- **Binary flag** – default_history was changed from object to bool because it represents a yes/no condition. and loan outcome to int

These changes make exploratory data analysis clearer, improve grouping and filtering speed, and prepare the columns for suitable encoding before training the model.

1. Categorical columns

```

df['gender'] = df['gender'].astype('category')
df['education'] = df['education'].astype('category')
df['home_ownership'] = df['home_ownership'].astype('category')
df['loan_purpose'] = df['loan_purpose'].astype('category')

```

```
# 2. Binary column
df['default_history'] = df['default_history'].map({'Yes': 1, 'No': 0}).astype('int64')
```

```
# 3. Target variable
```

```
df['loan_outcome'] = df['loan_outcome'].astype('int64')
```

```
# handle missing values
```

```
print(f"This dataset contains a total of {df.isnull().sum().sum()} missing values")
```

```
print("below is the distribution of missing values per column")
df.isna().sum()
```

This dataset contains a total of 0 missing values
below is the distribution of missing values per column

age	0
gender	0
education	0
income	0
employment_experience	0
home_ownership	0
loan_amount	0
loan_purpose	0
interest_rate	0
dti_ratio	0
credit_history_length	0
credit_score	0
default_history	0
loan_outcome	0
dtype: int64	

```
# Checking for duplicates
```

```
print(f"The dataset contains {df.duplicated().sum()} duplicated rows")
```

The dataset contains 0 duplicated rows

```
# select the data types
```

```
feature_cols = [col for col in df.columns if col not in  
['loan_outcome', 'loan_amount']]  
target_classification = 'loan_outcome'  
target_regression = 'loan_amount'
```

```
# Use feature_cols to select only FEATURES
```

```
num_cols = df[feature_cols].select_dtypes(include=['int64',  
'float64']).columns.tolist()  
cat_cols =  
df[feature_cols].select_dtypes(include=['category']).columns.tolist()
```

```
# display the columns
```

```
print("Numerical Feature columns:", num_cols)
```

```

print("Categorical Feature columns:", cat_cols)
print("Classification target:", target_classification)
print("Regression target:", target_regression)

Numerical Feaature columns: ['age', 'income', 'employment_experience',
'interest_rate', 'dti_ratio', 'credit_history_length', 'credit_score',
'default_history']
Categorical Feature columns: ['gender', 'education', 'home_ownership',
'loan_purpose']
Classification target: loan_outcome
Regression target: loan_amount

print("Identifying inconsistent labels in the categorical columns to
ensure via value counts")
for col in cat_cols:
    print(f"\n{col}:\n", df[col].value_counts(dropna=False))

Identifying inconsistent labels in the categorical columns to ensure
via value counts

gender:
  gender
male      24841
female    20159
Name: count, dtype: int64

education:
  education
BACHELOR    13399
ASSOCIATE    12028
HIGH SCHOOL  11972
MASTER       6980
DOCTORATE     621
Name: count, dtype: int64

home_ownership:
  home_ownership
RENT         23443
MORTGAGE     18489
OWN           2951
OTHER         117
Name: count, dtype: int64

loan_purpose:
  loan_purpose
EDUCATION    9153
MEDICAL      8548
VENTURE      7819
PERSONAL     7552
DEBTCONSOLIDATION 7145

```

```
HOMEIMPROVEMENT      4783
Name: count, dtype: int64
```

3.2.2 Univariate Analysis

Univariate analysis for Numerical variables

```
# define the colors
colors = ["skyblue", "salmon", "lightgreen", "gold", "plum",
"turquoise", "orange", "violet"]

# Use FEATURE columns
num_feature_cols = [col for col in num_cols if col not in
['loan_outcome', 'loan_amount']]

# Dynamic grid calculation
n_features = len(num_feature_cols)
n_cols = 3
n_rows = (n_features + n_cols - 1) // n_cols

plt.figure(figsize=(18, 4*n_rows))

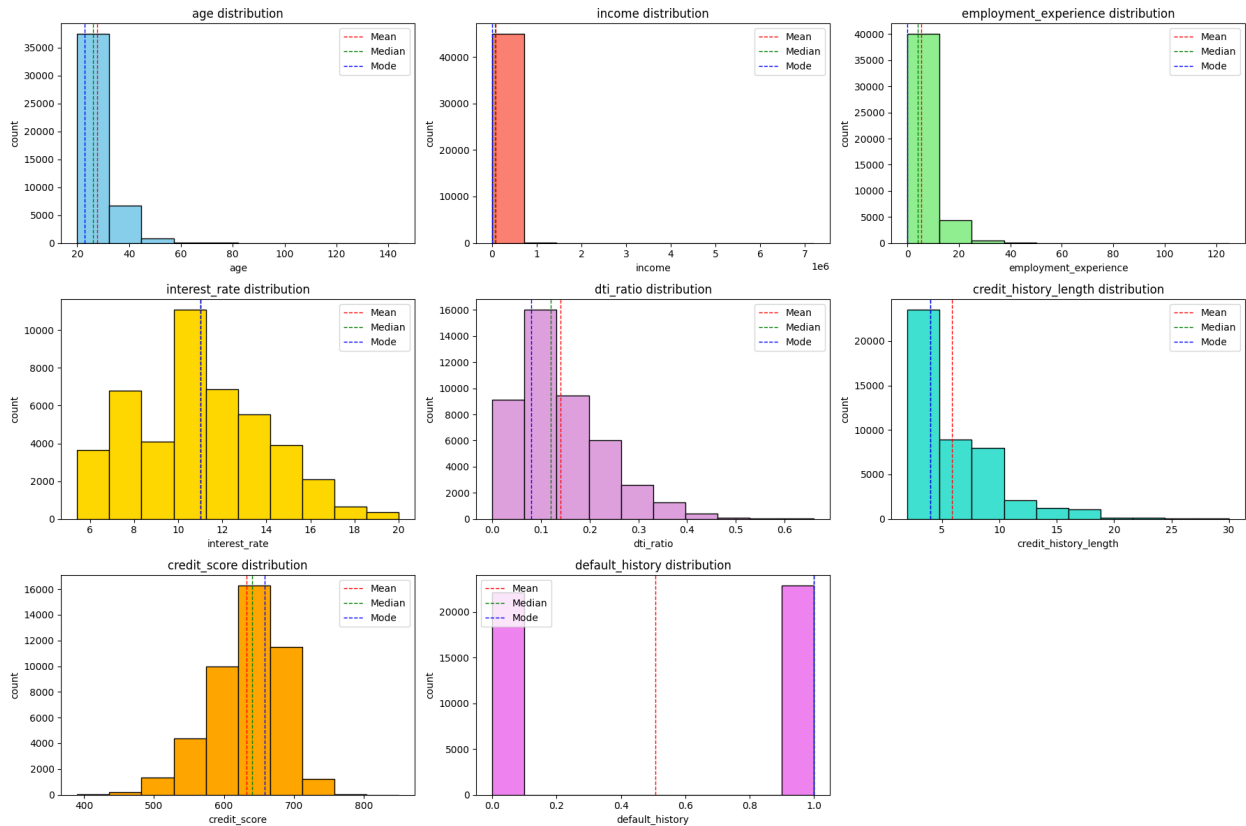
for i, col in enumerate(num_feature_cols, 1):
    plt.subplot(n_rows, n_cols, i)
    plt.hist(df[col], bins=10, color=colors[i-1], edgecolor="black")

    # Add mean and median sshow also in line
    plt.axvline(df[col].mean(), color='red', linestyle='dashed',
linewidth=1, label='Mean')
    plt.axvline(df[col].median(), color='green', linestyle='dashed',
linewidth=1, label='Median')

    # Safe mode plotting
    modes = df[col].mode()
    if not modes.empty:
        plt.axvline(modes[0], color='blue', linestyle='dashed',
linewidth=1, label='Mode')

    plt.title(f"{col} distribution")
    plt.xlabel(col)
    plt.ylabel("count")
    plt.legend()

# plot
plt.tight_layout()
plt.show()
```



1. Age - Most applicants are aged 20–30, with a few older outliers raising the mean. The mode is mid-20s, showing a youthful applicant pool. This skew may affect risk, as younger borrowers often have shorter credit histories and less financial stability. **Action:** The age groups should be binned, monitor younger applicants' risk and encode categories for modeling stability.
2. Income - Income is highly skewed, with most applicants earning very little. A few high earners inflate the mean. The mode is near zero. This imbalance suggests inequality **Action:** Apply log transformation, handle outliers, normalize distribution for fairer modeling outcomes.
3. Employment Experience - Most applicants have under 10 years of experience. The mean is higher than the median due to a few long careers. Mode is around 5 years. This reflects early career stages and may correlate with lower income and credit history length. **Action:** Bining experience levels to align with age, simplify categories for clearer model interpretation.
4. Loan Outcome - Loan outcome is binary, mostly 0 (negative). The mean is low, showing class imbalance. **Action:** Apply resampling or class weights, balance target variable and prevent biased predictions.
5. Interest Rate - Interest rates are evenly distributed between 6% and 14%. Mean, median, and mode align, showing no skew. This variable is stable and can be used directly in modeling without transformation.

6. DTI Ratio - DTI is right-skewed, with most applicants having low debt relative to income. A few high ratios raise the mean. Mode is near 0.1. High DTI may signal risk **Action:** Cap extreme values, scale ratios, highlight high-DTI cases for risk monitoring.
7. Credit History Length- - Most applicants have short credit histories of <10 years. The mean is higher than the median due to a few long records. Mode is around 5 years. Short histories may affect creditworthiness and risk assessment. **Action:** Bin into short, medium, long, encode categories, assess impact on loan outcome.
8. Credit Score - Credit scores are symmetric, mostly between 600 and 700. Mean, median, and mode align. This balanced distribution makes credit score a strong, stable predictor for loan outcomes. **Action:** Use directly, consider thresholds and retain raw values as strong predictor.

Univariate analysis for categorical variables

```
# Dynamic grid
n_cats = len(cat_cols)
n_cols = 2
n_rows = (n_cats + n_cols - 1) // n_cols

plt.figure(figsize=(12, 4*n_rows))

# Bar plots
for i, col in enumerate(cat_cols, 1):
    plt.subplot(n_rows, n_cols, i)

    # Get value counts
    value_counts = df[col].value_counts()

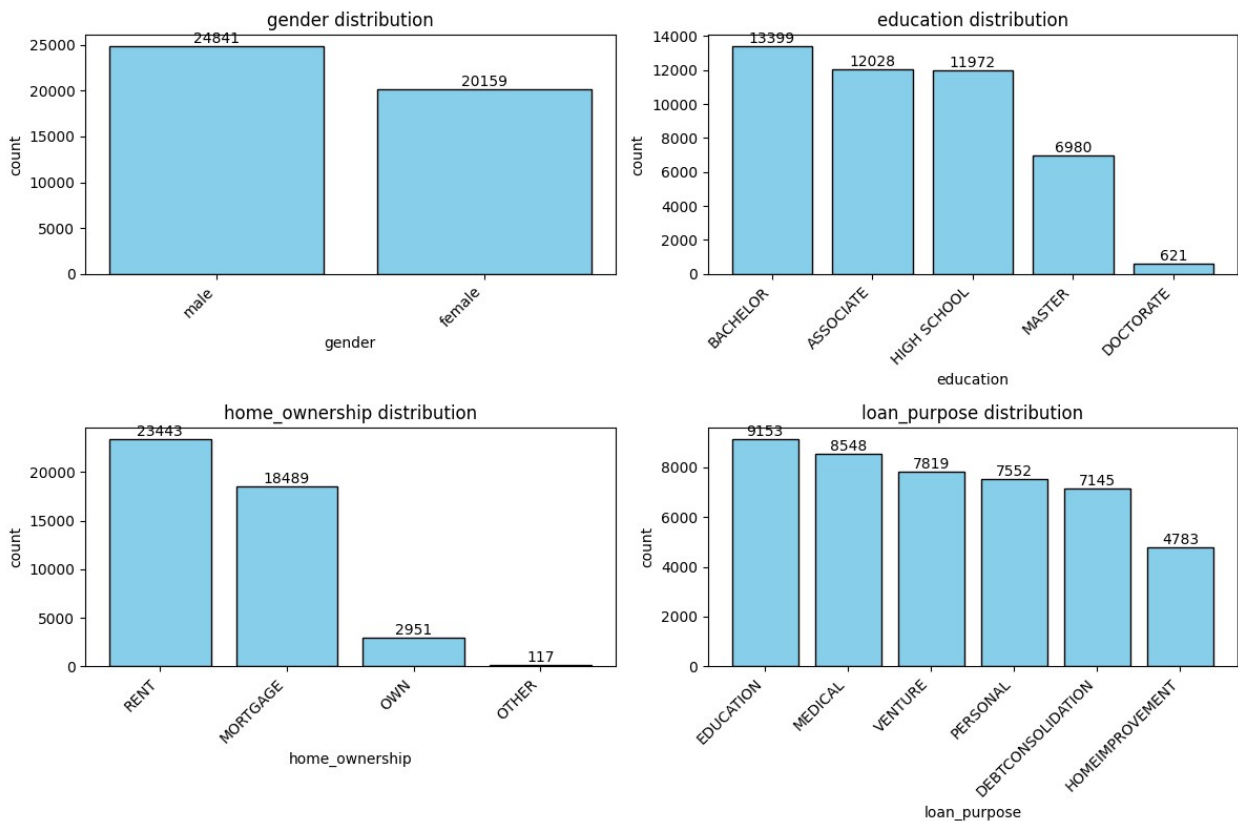
    # Plot
    bars = plt.bar(range(len(value_counts)), value_counts.values,
                    color="skyblue", edgecolor="black")

    # Set x-ticks with rotation for long labels
    plt.xticks(range(len(value_counts)), value_counts.index,
                rotation=45, ha='right')

    plt.title(f"{col} distribution")
    plt.xlabel(col)
    plt.ylabel("count")

    # Add count labels on top bars
    for bar in bars:
        height = bar.get_height()
        plt.text(bar.get_x() + bar.get_width()/2., height,
                 f'{int(height)}', ha='center', va='bottom')
```

```
plt.tight_layout()
plt.show()
```



1. **Gender** - The gender distribution shows whether applicants are balanced across male and female categories. If one group dominates, it may introduce bias in modeling. Since gender is binary, no transformation is needed, but imbalance should be noted. **Action:** Encode as binary, check fairness and monitor bias in predictive modeling.
2. **Education** - Education levels reveal applicant diversity. If most applicants fall into one category e.g., "Bachelor", rare categories may need grouping into "Other." This variable can highlight socioeconomic differences that influence loan outcomes. **Action:** Group rare categories, encode levels and explore correlation with income and loan outcome.
3. **Home Ownership** - Home ownership status, owner vs renter often reflects financial stability. A dominant category indicates imbalance, while differences across groups may be predictive of default risk. **Action:** Encode ownership tiers, group rare categories and assess impact on default risk.
4. **Loan Purpose** - Loan purpose typically has multiple categories e.g., personal, business, education. Some purposes may dominate, while others are rare. Rare categories should be grouped to avoid noise. This variable is important for risk

segmentation, as purpose often correlates with repayment behavior. **Action:** Group rare purposes, encode categories and analyze default rates by loan purpose.

5. Default History - Default history is binary (yes/no). Most applicants likely have “no default,” but the presence of “yes” cases is critical for modeling. This variable is a strong predictor of loan outcome. **Action:** Encode as binary, prioritize in modeling and highlight default history as key risk feature.

3.2.3 Bivariate Analysis

Bivariate analysis for Numerical variables vs Target variable

```
# Dynamic grid
n_features = len(num_feature_cols)
n_cols = 3
n_rows = (n_features + n_cols - 1) // n_cols

plt.figure(figsize=(15, 4*n_rows))

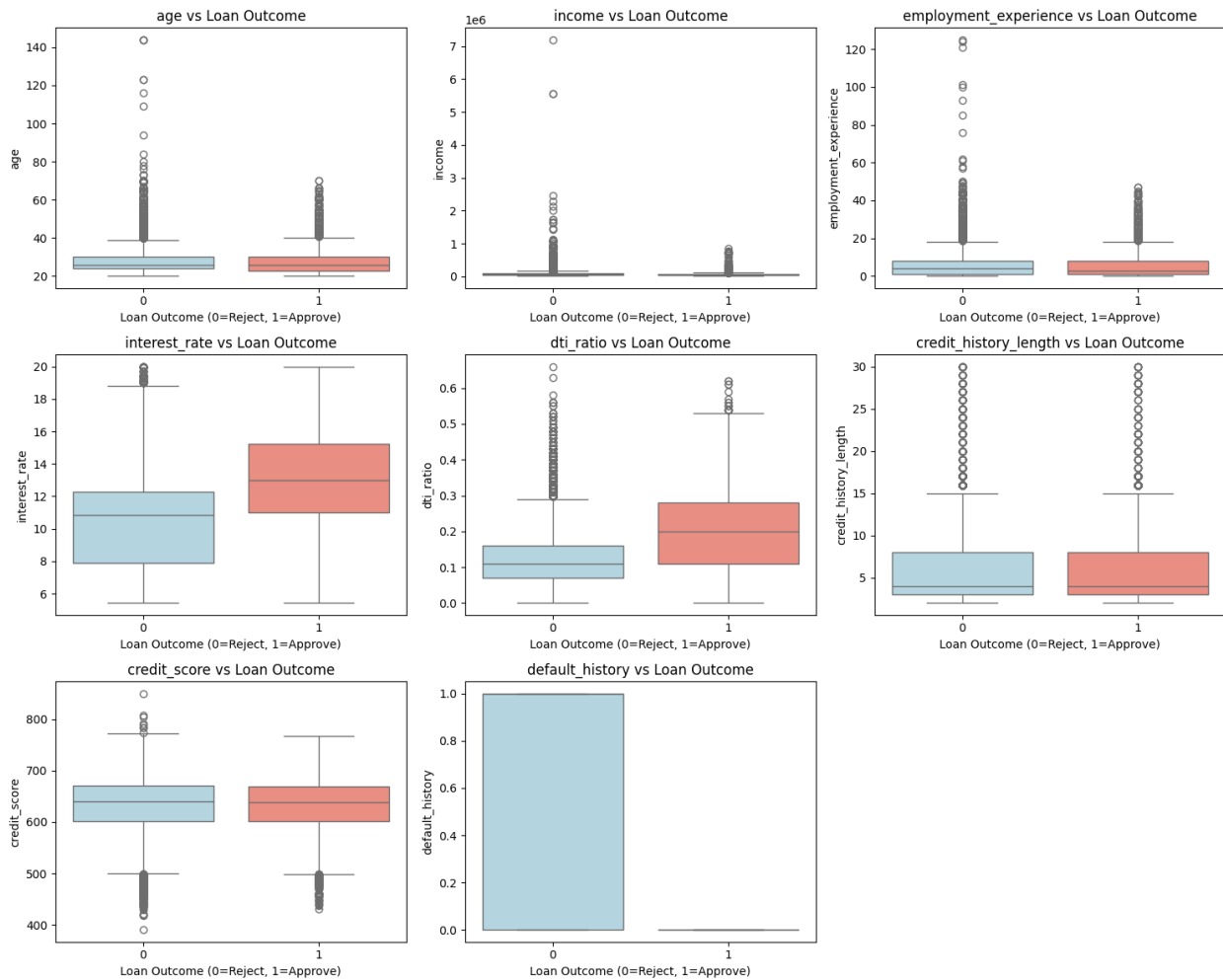
for i, col in enumerate(num_feature_cols, 1):
    plt.subplot(n_rows, n_cols, i)

    # Boxplot for binary classification
    sns.boxplot(x="loan_outcome", y=col, data=df,
                palette=["lightblue", "salmon"])

    plt.title(f"{col} vs Loan Outcome", fontsize=12)
    plt.xlabel("Loan Outcome (0=Reject, 1=Approve)", fontsize=10)
    plt.ylabel(col, fontsize=10)

    # Add statistical annotation
    rejected = df[df['loan_outcome'] == 0][col]
    approved = df[df['loan_outcome'] == 1][col]

plt.tight_layout()
plt.show()
```

1. Age - Successful applicants tend to be slightly older, while younger ones show more variability and outliers. Age may weakly influence loan approval. **Action:** binning age groups and dropping if redundant with experience or credit history.
2. Income - Higher income clearly aligns with successful outcomes. Defaults cluster around lower income levels, showing income is a strong predictor of loan performance. **Action:** Apply log transformation, retain as a key feature for modeling.
3. Employment Experience - Applicants with more experience show better outcomes. Defaults are common among those with shorter work histories, reflecting early career risk. **Action:** Bin experience levels, drop if age or credit history already included.
4. Loan Amount - Defaults are more frequent with larger loan amounts. Successful loans tend to be moderate, suggesting risk increases with loan size. **Action:** Apply log scaling, flag large loans and retain for risk modeling.
5. Interest Rate - Successful loans have slightly lower interest rates. Distribution is tight, but shows some separation between outcomes. **Action:** Keep raw, use directly in modeling; consider interaction with credit score.

6. DTI Ratio - Higher DTI ratios are linked to defaults. Successful applicants maintain lower debt relative to income, making DTI a strong risk signal. **Action:** Cap extreme values. scale and retain as a key predictor.
7. Credit History Length - Longer credit histories correlate with successful outcomes. Defaults cluster around shorter histories, showing credit maturity matters. **Action:** Bin into short/medium/long, retain or combine with experience.
8. Credit Score - Higher scores are clearly associated with successful outcomes. Strong separation makes this one of the most powerful predictors. **Action:** Use directly retain as a core feature, consider threshold-based segmentation.

Bivariate analysis for categorical variables vs Target variable

```
# Include default_history in categorical analysis
categorical_features = cat_cols + ['default_history']

# Dynamic grid
n_features = len(categorical_features)
n_cols = 2
n_rows = (n_features + n_cols - 1) // n_cols

plt.figure(figsize=(15, 4*n_rows))

for i, col in enumerate(categorical_features, 1):
    plt.subplot(n_rows, n_cols, i)

    # For binary default_history, ensure proper labeling
    if col == 'default_history':
        # Map 1/0 back to Yes/No for visualization
        temp_df = df.copy()
        temp_df['default_history'] =
temp_df['default_history'].map({1: 'Yes', 0: 'No'})
        plot_df = temp_df
    else:
        plot_df = df

    # Create countplot
    ax = sns.countplot(x=col, hue="loan_outcome", data=plot_df,
                        palette={0: "lightcoral", 1: "lightgreen"})

    plt.title(f"{col} vs Loan Outcome", fontsize=12)
    plt.xlabel(col, fontsize=10)
    plt.ylabel("Count", fontsize=10)

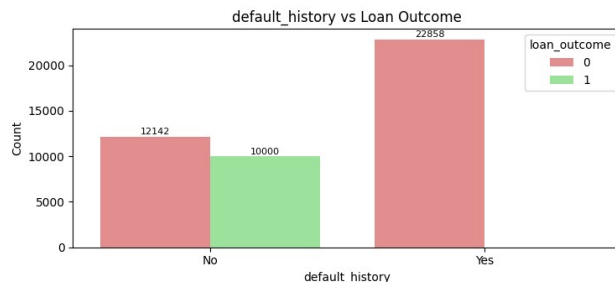
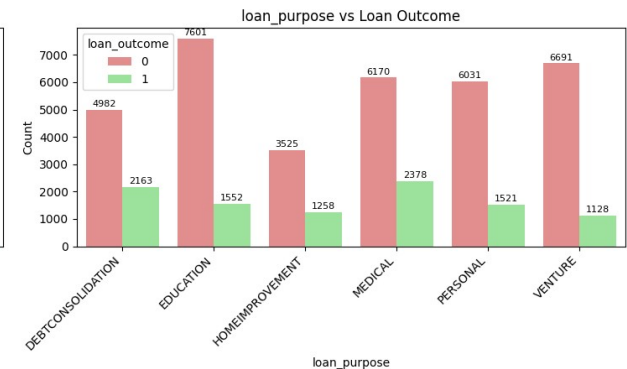
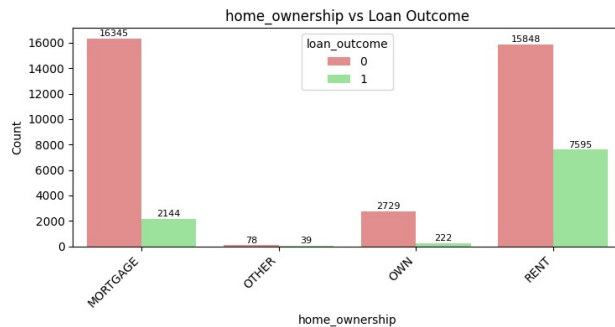
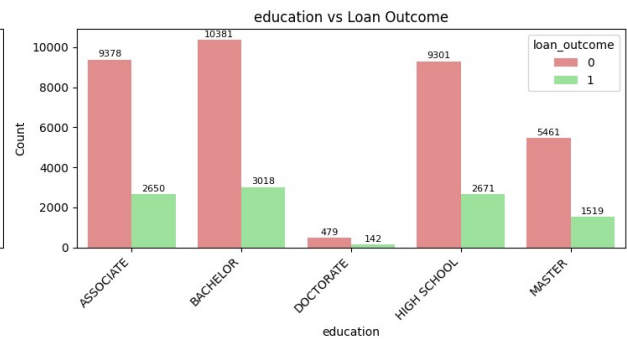
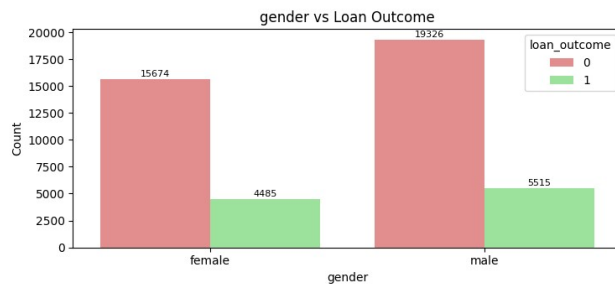
    # Smart rotation based on number of categories
    n_categories = len(plot_df[col].unique())
    rotation = 45 if n_categories > 3 else 0
    plt.xticks(rotation=rotation, ha='right' if rotation > 0 else
'center')
```

```

# Add percentage annotations
total_counts = plot_df[col].value_counts()
for p in ax.patches:
    height = p.get_height()
    if height > 0:
        ax.text(p.get_x() + p.get_width()/2., height + 50,
                f'{int(height)}', ha='center', va='bottom',
                fontsize=8)

plt.tight_layout()
plt.show()

```



```

# Add chi-square test results
for col in categorical_features:
    contingency_table = pd.crosstab(df[col], df['loan_outcome'])
    chi2, p, dof, expected = chi2_contingency(contingency_table)
    print(f"{col}:  $\chi^2$ = {chi2:.2f}, p={p:.4f}, {'Significant' if p < 0.05 else 'Not significant'}")

```

```
gender:  $\chi^2=0.01$ ,  $p=0.9143$ , Not significant  
education:  $\chi^2=2.02$ ,  $p=0.7328$ , Not significant  
home_ownership:  $\chi^2=2989.51$ ,  $p=0.0000$ , Significant  
loan_purpose:  $\chi^2=909.65$ ,  $p=0.0000$ , Significant  
default_history:  $\chi^2=13270.29$ ,  $p=0.0000$ , Significant
```

1. Gender - Male applicants dominate both loan outcomes, but the success rate appears similar across genders. No strong predictive separation is visible. **Action:** Encode as binary, retain for fairness checks, drop if model shows no performance gain.
2. Education - Bachelor and High School levels dominate. Higher education such Master, Doctorate shows slightly better outcomes, but sample sizes are small. **Action:** Group rare categories, encode ordinally and explore interaction with income and loan purpose.
3. Home Ownership - Mortgage and Own categories show better outcomes than Rent. Renters have higher default counts, suggesting financial instability. **Action:** Encode ownership tiers, group "Other" and retain as a risk indicator.
4. Loan Purpose - Debt consolidation and personal loans show more defaults. Education loans perform better, suggesting purpose influences repayment behavior. **Action:** Group rare purposes; encode categories and monitor high-risk types like debt and personal loans.
5. Default History - Applicants with prior defaults show significantly more negative outcomes. This is the strongest categorical predictor of loan risk. **Action:** Encode as binary, prioritize in modeling and use for early risk flagging.

3.2.4 Multivariate Analysis

```
# Use numerical features only (exclude targets)
num_feature_cols = [col for col in df.select_dtypes(include=['int64',
'float64']).columns
                    if col not in ['loan_outcome', 'loan_amount']]

# Compute correlation
corr_matrix = df[num_feature_cols].corr()

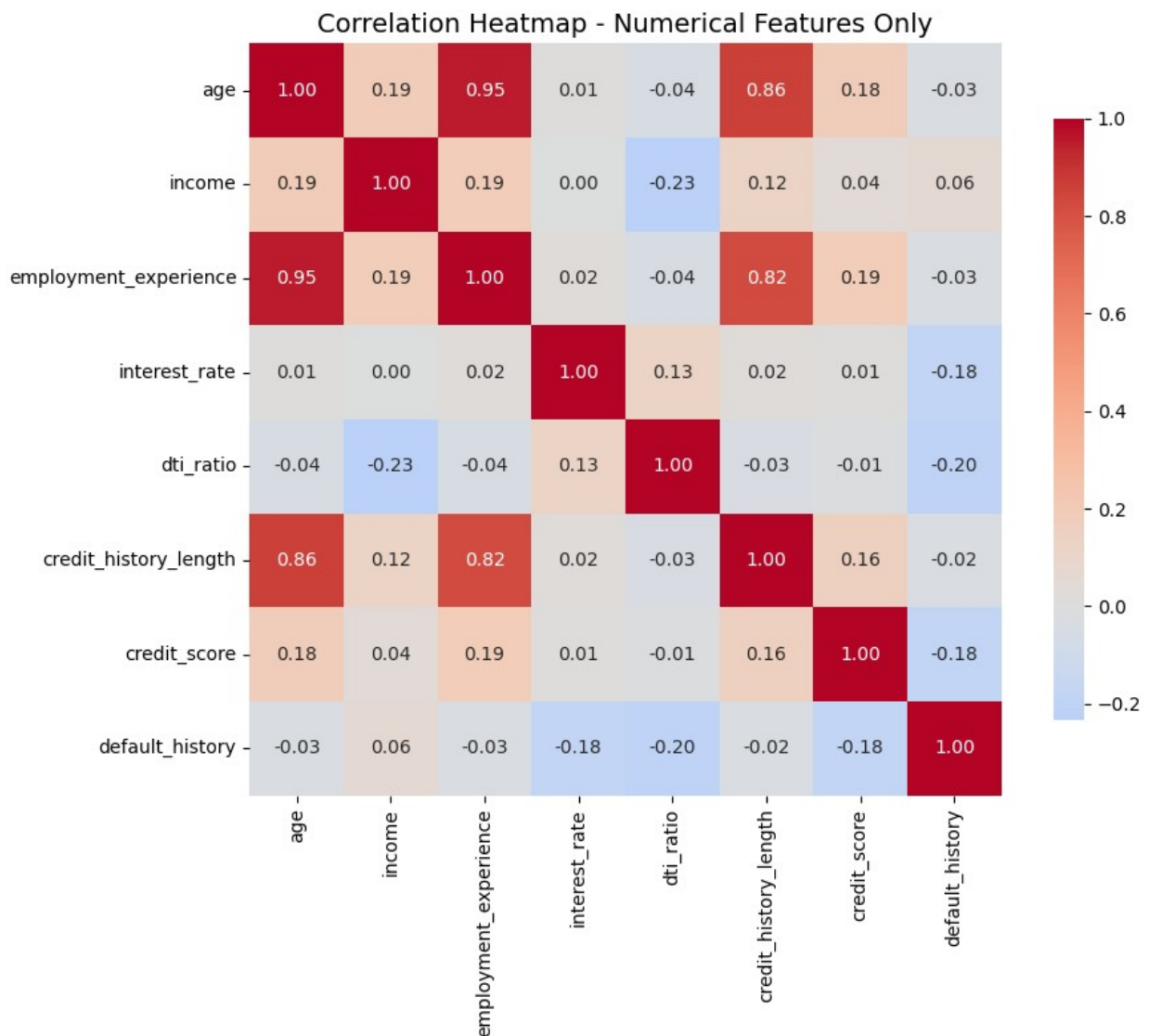
# Plot heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", center=0,
fmt='.2f',
            square=True, cbar_kws={"shrink": 0.8})
plt.title("Correlation Heatmap - Numerical Features Only",
fontsize=14)
plt.tight_layout()
plt.show()

# Interpret results
```

```

high_corr_pairs = []
for i in range(len(corr_matrix.columns)):
    for j in range(i+1, len(corr_matrix.columns)):
        corr_value = abs(corr_matrix.iloc[i, j])
        if corr_value > 0.7:
            col1, col2 = corr_matrix.columns[i],
corr_matrix.columns[j]
            high_corr_pairs.append((col1, col2, corr_matrix.iloc[i,
j]))
            print(f"{col1} vs {col2}: r = {corr_matrix.iloc[i,
j]:.3f}")

```



```

age vs employment_experience: r = 0.954
age vs credit_history_length: r = 0.862
employment_experience vs credit_history_length: r = 0.824

```

```

# checking n=signifact numerical features
for col in num_feature_cols:
    group0 = df[df['loan_outcome'] == 0][col]
    group1 = df[df['loan_outcome'] == 1][col]

    # Remove NaN if any
    group0 = group0.dropna()
    group1 = group1.dropna()

    f_stat, p_value = f_oneway(group0, group1)
    print(f"{col}: F={f_stat:.2f}, p={p_value:.4f}, {'Significant' if
p_value < 0.05 else 'Not significant'}")

age: F=20.76, p=0.0000, Significant
income: F=845.53, p=0.0000, Significant
employment_experience: F=18.88, p=0.0000, Significant
interest_rate: F=5574.45, p=0.0000, Significant
dti_ratio: F=7824.79, p=0.0000, Significant
credit_history_length: F=9.93, p=0.0016, Significant
credit_score: F=2.63, p=0.1048, Not significant
default_history: F=18824.73, p=0.0000, Significant

# VIF ANALYSIS ro check Multicollinearity
# Prepare data for VIF (add constant)
X_vif = df[num_feature_cols].copy()
X_vif = add_constant(X_vif)

# Calculate VIF
vif_data = pd.DataFrame()
vif_data["Feature"] = X_vif.columns
vif_data["VIF"] = [variance_inflation_factor(X_vif.values, i)
                    for i in range(len(X_vif.columns))]

print(vif_data.sort_values("VIF", ascending=False))

# Rule of thumb: VIF > 10 indicates severe multicollinearity
high_vif = vif_data[vif_data['VIF'] > 10]
if not high_vif.empty:
    print(f"\nWARNING: {len(high_vif)} features with VIF > 10:")
    print(high_vif[['Feature', 'VIF']])

```

	Feature	VIF
0	const	374.83
1	age	14.11
3	employment_experience	11.26
6	credit_history_length	3.92
5	dti_ratio	1.11
8	default_history	1.11
2	income	1.11
7	credit_score	1.07

4	interest_rate	1.05
---	---------------	------

WARNING: 3 features with VIF > 10:

	Feature	VIF
0	const	374.83
1	age	14.11
3	employment_experience	11.26

1. Age, Credit History Length and Employment Experience are highly correlated with severe multicollinearity (VIF > 10). **Action:** Keep Credit History Length (most direct financial indicator, lower VIF=3.92). Drop Age (VIF=14.11) and Employment Experience (VIF=11.26).
2. Gender shows no statistical significance ($\chi^2=0.01$, $p=0.9143$). **Action:** Drop Gender.
3. Education shows no statistical significance ($\chi^2=2.02$, $p=0.7328$). **Action:** Drop Education.
4. Income and Loan Amount are skewed. **Action:** Apply log transformation after train-test split.
5. DTI Ratio is skewed and contains outliers. **Action:** Cap outliers using IQR method, then apply log transformation.
6. Class Imbalance Detected: 77.8% rejected vs 22.2% approved loans. **Action:** Use `class_weight='balanced'` in classification models.
7. Default History, Home Ownership, Loan Purpose show high statistical significance ($p=0.0000$). **Action:** Keep all.
8. Credit Score is borderline significant ($p=0.1048$) but business critical. **Action:** Keep for business logic.
9. DTI Ratio and Interest Rate have highest feature importance. **Action:** Prioritize these features.

Features to Keep: Credit History Length, Income, DTI Ratio, Interest Rate, Credit Score, Default History, Home Ownership, Loan Purpose and Features to Drop: Age, Employment Experience, Gender, Education

3.2.6 Data Preprocessing

```
# List of columns to keep for modeling
selected_cols = [
    "credit_score",
    "credit_history_length",
    "income",
    "loan_amount",
    "interest_rate",
    "dti_ratio",
```

```

    "default_history",
    "home_ownership",
    "loan_purpose",
    "loan_outcome"
]

# Create new DataFrame
data = df[selected_cols].copy()

print("\nTargets:")
print("- loan_outcome (classification)")
print("- loan_amount (regression)")
data.head()

```

Targets:

- loan_outcome (classification)
- loan_amount (regression)

	credit_score	credit_history_length	income	\
0	561	3.00	71948.00	
1	504	2.00	12282.00	
2	635	3.00	12438.00	
3	675	2.00	79753.00	
4	586	4.00	66135.00	

	loan_amount	interest_rate	dti_ratio	\
0	35000.00	16.02	0.49	
1	1000.00	11.14	0.08	
2	5500.00	12.87	0.44	
3	35000.00	15.23	0.44	
4	35000.00	14.27	0.53	

	default_history	home_ownership	loan_purpose	loan_outcome
0	0	RENT	PERSONAL	1
1	1	OWN	EDUCATION	0
2	0	MORTGAGE	MEDICAL	1
3	0	RENT	MEDICAL	1
4	0	RENT	MEDICAL	1

Apply basic business rule clipping

```

business_rules = {
    'income': (0, 10_000_000),
    'dti_ratio': (0, 1.0),
    'credit_score': (300, 850),
    'interest_rate': (0, 30),
}

for col, (min_val, max_val) in business_rules.items():
    if col in data.columns:

```



```

        data[col] = data[col].clip(min_val, max_val)
        print(f"Clipped {col} to [{min_val}, {max_val}]")

Clipped income to [0, 100000000]
Clipped dti_ratio to [0, 1.0]
Clipped credit_score to [300, 850]
Clipped interest_rate to [0, 30]

# Show frequency to identify rare columns
for col in ["home_ownership", "loan_purpose"]:
    print(data[col].value_counts())

home_ownership
RENT      23443
MORTGAGE  18489
OWN       2951
OTHER     117
Name: count, dtype: int64
loan_purpose
EDUCATION      9153
MEDICAL        8548
VENTURE        7819
PERSONAL       7552
DEBTCONSOLIDATION 7145
HOMEIMPROVEMENT 4783
Name: count, dtype: int64

```

Grouping of Rare Categories in categorical variables

1. Home Ownership – Own has 2,951 entries and Other has only 117 entries. Since “Other” is very rare and “Own” is relatively small compared to Rent and Mortgage, combining Own and Other into a single category will be good and will be named as **OWN/OTHER**.
2. Loan Purpose – Education has 9,153, Medical has 8,548, Venture has 7,819, Personal has 7,552, Debt Consolidation has 7,145, and Home Improvement has 4,783 all have several thousand entries. None of these categories are rare enough to warrant grouping, so they will be kept as separate categories for modeling.

```

# COMBINING RARE CATEGORIES

# HOME OWNERSHIP
data['home_ownership'] = data['home_ownership'].replace({
    'OWN': 'OWN/OTHER',
    'OTHER': 'OWN/OTHER'
})
# preview
print(f'OWN/OTHER contains {(data["home_ownership"] ==
    "OWN/OTHER").sum()} entries')

```

OWN/OTHER contains 3068 entries

3.2.7 Data Branching

This project involves building three separate models. To support them, we will create three dataframes from the main dataset, each with the appropriate features and target variable:

1. `data_class`, for the classification task, where the target variable is `loan_outcome`. Other target variables will be dropped.
2. `data_regres1`, for the regression task to predict the loan amount issued.
3. `data_regres2`, for the clustering to predict the interest rate as part of the risk-based analysis which will be intergrated with central bank Kesonic formula.

```
### **3.2.6 Data Preprocessing**
```

```
data.columns
```

```
Index(['credit_score', 'credit_history_length', 'income',  
      'loan_amount',  
      'interest_rate', 'dti_ratio', 'default_history',  
      'home_ownership',  
      'loan_purpose', 'loan_outcome'],  
      dtype='object')
```

```
# 1. Classification data (loan_outcome as target)
```

```
class_features = [  
    'credit_score',  
    'credit_history_length',  
    'income',  
    'dti_ratio',  
    'default_history',  
    'home_ownership',  
    'loan_purpose',  
    'loan_outcome'  
]
```

```
# Check which features actually exist
```

```
existing_features = [col for col in class_features if col in  
data.columns]
```

```
data_class = data[existing_features].copy()
```

```
# preview
```

```
data_class.head()
```

	credit_score	credit_history_length	income \
0	561	3.00	71948.00
1	504	2.00	12282.00
2	635	3.00	12438.00
3	675	2.00	79753.00
4	586	4.00	66135.00

	dti_ratio	default_history	home_ownership	loan_purpose	\
0	0.49	0	RENT	PERSONAL	
1	0.08	1	OWN/OTHER	EDUCATION	
2	0.44	0	MORTGAGE	MEDICAL	
3	0.44	0	RENT	MEDICAL	
4	0.53	0	RENT	MEDICAL	

	loan_outcome
0	1
1	0
2	1
3	1
4	1

Data for regression to predict loan amount for APPROVED loans

```
regression_features = [
    'credit_score',
    'credit_history_length',
    'income',
    'dti_ratio',
    'default_history',
    'home_ownership',
    'loan_purpose',
    'loan_amount'
]
```

Check which features actually exist

```
existing_features = [col for col in regression_features if col in
data.columns]
```

Create dataset with ALL data first (we'll filter later)

```
data_regres1 = data[existing_features].copy()
```

Now filter to APPROVED LOANS ONLY for regression training

```
approved_mask = data['loan_outcome'] == 1
data_regres1_approved = data_regres1[approved_mask].copy()
```

preview

```
data_regres1_approved.head()
```

	credit_score	credit_history_length	income	\
0	561	3.00	71948.00	
2	635	3.00	12438.00	
3	675	2.00	79753.00	
4	586	4.00	66135.00	
5	532	2.00	12951.00	

	dti_ratio	default_history	home_ownership	loan_purpose	\
0	0.49	0	RENT	PERSONAL	
2	0.44	0	MORTGAGE	MEDICAL	

3	0.44	0	RENT	MEDICAL
4	0.53	0	RENT	MEDICAL
5	0.19	0	OWN/OTHER	VENTURE

	loan_amount
0	35000.00
2	5500.00
3	35000.00
4	35000.00
5	2500.00

Data for clustering and interest rate prediction

```
interest_features = [
    'credit_score',
    'credit_history_length',
    'income',
    'dti_ratio',
    'default_history',
    'loan_amount',
    'home_ownership',
    'loan_purpose',
    'interest_rate'
]
```

Filter to approved loans only

```
approved_mask = data['loan_outcome'] == 1
data_int = data[approved_mask][interest_features].copy()
```

preview

```
data_int.head()
```

	credit_score	credit_history_length	income	\
0	561	3.00	71948.00	
2	635	3.00	12438.00	
3	675	2.00	79753.00	
4	586	4.00	66135.00	
5	532	2.00	12951.00	

	dti_ratio	default_history	loan_amount
home_ownership \			
0	0.49	0	35000.00
RENT			
2	0.44	0	5500.00
MORTGAGE			
3	0.44	0	35000.00
RENT			
4	0.53	0	35000.00
RENT			
5	0.19	0	2500.00
OWN/OTHER			

	loan_purpose	interest_rate
0	PERSONAL	16.02
2	MEDICAL	12.87
3	MEDICAL	15.23
4	MEDICAL	14.27
5	VENTURE	7.14

```
data_int.dtypes
```

credit_score	int64
credit_history_length	float64
income	float64
dti_ratio	float64
default_history	int64
loan_amount	float64
home_ownership	category
loan_purpose	category
interest_rate	float64
dtype:	object

4.0 Modelling

The goal of this stage is to build predictive models that address the project objectives. Three analyses will be carried out:

- Classification - Predict if a loan application should be approved where loan_outcome is the target variable.
- Regression 1 - Predict the loan amount a client is likely to receive where loan_amount is the target variable.
- Regression 2 - predict the interest rate at which a given loan will attract and the target variable is interest_rate.

4.1 Classification Model For Approve/Reject

The classification analysis focuses on predicting whether a client qualifies for a loan. The target variable is loan_outcome, where 1 represents approval and 0 represents rejection. We will train and evaluate three models and all the models will use a test size of 20% and random state 42, the models include:

- Logistic Regression
- Random Forest Classifier
- XGBoost Classifier

Model performance will be assessed using metrics suited for classification, including accuracy, precision, recall, F1-score, and ROC-AUC. The aim is to select a model that balances predictive accuracy with reliability in identifying eligible applicants.

```

# define X and y
y = data_class["loan_outcome"]
X = data_class.drop("loan_outcome", axis=1)

# Encoding categorical columns
X_encoded = pd.get_dummies(X, columns=['home_ownership',
'loan_purpose'], drop_first=True)

print(f"Original X shape: {X.shape}")
print(f"Encoded X shape: {X_encoded.shape}")

Original X shape: (45000, 7)
Encoded X shape: (45000, 12)

# train test split
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y,
test_size=0.2, random_state=42, stratify=y)

# Check the shapes
print(f"Training data shape: {X_train.shape}")
print(f"Testing data shape: {X_test.shape}")

Training data shape: (36000, 12)
Testing data shape: (9000, 12)

# Apply log transformations to skewed features income and dti_ratio
log_transform_cols = ['income', 'dti_ratio']

for col in log_transform_cols:
    if col in X_train.columns:
        # Apply loglp to training data
        X_train[col] = np.loglp(X_train[col])

        # Apply same transformation to test data
        X_test[col] = np.loglp(X_test[col])

# Applying IQR clipping to handle extreme values
def clip_iqr_columns(df, columns, iqr_multiplier=1.5):
    """Clip outliers using IQR method"""
    df_clipped = df.copy()

    for col in columns:
        if col in df.columns:
            Q1 = df[col].quantile(0.25)
            Q3 = df[col].quantile(0.75)
            IQR = Q3 - Q1

            lower_bound = Q1 - iqr_multiplier * IQR
            upper_bound = Q3 + iqr_multiplier * IQR

            # Count outliers before clipping

```

```

        outliers_low = (df[col] < lower_bound).sum()
        outliers_high = (df[col] > upper_bound).sum()
        total_outliers = outliers_low + outliers_high

        if total_outliers > 0:
            print(f" {col}: Clipping {total_outliers} outliers")
            df_clipped[col] = df[col].clip(lower_bound,
upper_bound)

    return df_clipped

# Apply to numerical columns
numerical_cols = ['credit_score', 'income', 'dti_ratio',
'credit_history_length']

# Clipping outliers in training data
X_train = clip_iqr_columns(X_train, numerical_cols)

# Applying same clipping bounds to test data
for col in numerical_cols:
    if col in X_train.columns:
        # Use training data bounds for test data
        train_min = X_train[col].min()
        train_max = X_train[col].max()
        X_test[col] = X_test[col].clip(train_min, train_max)

credit_score: Clipping 406 outliers
income: Clipping 630 outliers
dti_ratio: Clipping 407 outliers
credit_history_length: Clipping 1092 outliers

# Apply StandardScaler to numerical features
scale_cols = ['credit_score', 'income', 'dti_ratio',
'credit_history_length']

# Initialize the scaler
scaler = StandardScaler()

# Fit on TRAIN data only, transform both train and test
X_train[scale_cols] = scaler.fit_transform(X_train[scale_cols])
X_test[scale_cols] = scaler.transform(X_test[scale_cols])

```

4.1.1 Logistic Regression

```

# train log_clf
log_clf = LogisticRegression(max_iter=1000, class_weight='balanced',
random_state=42)
log_clf.fit(X_train, y_train)

```

```
LogisticRegression(class_weight='balanced', max_iter=1000,  
random_state=42)
```

```
# predict
```

```
y_pred_log = log_clf.predict(X_test)
```

```
# probability
```

```
y_proba_log = log_clf.predict_proba(X_test)[:, 1]
```

```
# Hard predictions
```

```
y_pred_log = log_clf.predict(X_test)
```

```
# Probabilities
```

```
y_proba_log = log_clf.predict_proba(X_test)[:, 1]
```

```
# Classification report
```

```
print("Classification Report:")
```

```
print(classification_report(y_test, y_pred_log))
```

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.79	0.87	7000
1	0.56	0.91	0.69	2000
accuracy			0.82	9000
macro avg	0.76	0.85	0.78	9000
weighted avg	0.88	0.82	0.83	9000

4.1.2 Random Forest Classifier

```
# train log_clf
```

```
rf_clf = RandomForestClassifier(n_estimators=100,
```

```
class_weight='balanced', random_state=42)
```

```
rf_clf.fit(X_train, y_train)
```

```
RandomForestClassifier(class_weight='balanced', random_state=42)
```

```
# hard predictions
```

```
y_pred_rf = rf_clf.predict(X_test)
```

```
# probabilities
```

```
y_proba_rf = rf_clf.predict_proba(X_test)[:, 1]
```

```
# Hard predictions
```

```
y_pred_rf = rf_clf.predict(X_test)
```

```
# Probabilities
```

```
y_proba_rf = rf_clf.predict_proba(X_test)[:, 1]
```



```
# Classification report
print("Classification Report:")
print(classification_report(y_test, y_pred_rf))
```

Classification Report:

	precision	recall	f1-score	support
0	0.91	0.97	0.94	7000
1	0.85	0.66	0.74	2000
accuracy			0.90	9000
macro avg	0.88	0.81	0.84	9000
weighted avg	0.90	0.90	0.89	9000

Random Forest Classifier Hyperparameter tuning

```
# Define the parameter grid
param_dist = {
    'n_estimators': np.arange(100, 500, 50),
    'max_depth': [None, 5, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2', None],
    'class_weight': ['balanced', 'balanced_subsample']
}

# Initialize the Random Forest
rf_clf_tuned = RandomForestClassifier(random_state=42)

# Randomized Search
rf_random = RandomizedSearchCV(
    estimator=rf_clf_tuned,
    param_distributions=param_dist,
    n_iter=50,
    cv=3,
    verbose=2,
    random_state=42,
    n_jobs=-1
)

# Fit the model
rf_random.fit(X_train, y_train)

# Best parameters
print("Best Parameters:", rf_random.best_params_)

# Evaluate on test set
best_rf = rf_random.best_estimator_
y_pred_best_rf = best_rf.predict(X_test)
y_proba_best_rf = best_rf.predict_proba(X_test)[:, 1]
```

```
# evaluate metrics
print("Classification Report (Tuned Random Forest):")
print(classification_report(y_test, y_pred_best_rf))
print("ROC-AUC:", roc_auc_score(y_test, y_proba_best_rf))
```

Fitting 3 folds for each of 50 candidates, totalling 150 fits
Best Parameters: {'n_estimators': np.int64(400), 'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'log2', 'max_depth': 30, 'class_weight': 'balanced_subsample'}

Classification Report (Tuned Random Forest):

	precision	recall	f1-score	support
0	0.91	0.97	0.94	7000
1	0.85	0.67	0.75	2000
accuracy			0.90	9000
macro avg	0.88	0.82	0.84	9000
weighted avg	0.90	0.90	0.89	9000

ROC-AUC: 0.9549735000000001

```
# Custom threshold evaluation
threshold = 0.35
y_pred_custom = (y_proba_best_rf >= threshold).astype(int)

print("\nClassification Report (Threshold = 0.35):")
print(classification_report(y_test, y_pred_custom))
print("ROC-AUC:", roc_auc_score(y_test, y_proba_best_rf))
```

Classification Report (Threshold = 0.35):

	precision	recall	f1-score	support
0	0.94	0.91	0.93	7000
1	0.72	0.80	0.76	2000
accuracy			0.89	9000
macro avg	0.83	0.86	0.84	9000
weighted avg	0.89	0.89	0.89	9000

ROC-AUC: 0.9549735000000001

4.1.3 XGboost Classifier

```
# calculate the scale pos weight
scale_pos_weight = len(y_train[y_train==0])/len(y_train[y_train==1])

# Train XGboost
Xgb_clf = XGBClassifier(
    n_estimators = 100,
    learning_rate = 0.1,
```

```

    max_depth = 5,
    scale_pos_weight = scale_pos_weight,
    random_state = 42,
    eval_metric = 'logloss'
)
Xgb_clf.fit(X_train, y_train)

XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None,
              early_stopping_rounds=None,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, feature_weights=None, gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=0.1,
              max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=5, max_leaves=None,
              min_child_weight=None, missing=nan,
              monotone_constraints=None,
              multi_strategy=None, n_estimators=100, n_jobs=None,
              num_parallel_tree=None, ...)

# hard predictions
y_pred_xg = Xgb_clf.predict(X_test)

# probabilites
y_proba_xg = Xgb_clf.predict_proba(X_test)[:, 1]

# Hard predictions
y_pred_xg = Xgb_clf.predict(X_test)

# Probabilities
y_proba_xg = Xgb_clf.predict_proba(X_test)[:, 1]

# Classification report
print("Classification Report:")
print(classification_report(y_test, y_pred_xg))

```

```

Classification Report:

```

	precision	recall	f1-score	support
0	0.97	0.83	0.89	7000
1	0.60	0.92	0.72	2000
accuracy			0.85	9000
macro avg	0.79	0.87	0.81	9000
weighted avg	0.89	0.85	0.85	9000

XGBoost Classifier Hypeparameter tuning

```
# Calculate scale_pos_weight
scale_pos_weight = len(y_train[y_train==0]) / len(y_train[y_train==1])

# Define the parameter grid
param_dist = {
    'n_estimators': np.arange(100, 500, 50),
    'learning_rate': [0.01, 0.05, 0.1, 0.2],
    'max_depth': [3, 5, 7, 10],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
    'gamma': [0, 0.1, 0.2, 0.5],
    'min_child_weight': [1, 3, 5],
    'scale_pos_weight': [scale_pos_weight]
}

# Initialize the XGBClassifier
xgb_clf_tuned = XGBClassifier(
    random_state=42,
    eval_metric='logloss',
    use_label_encoder=False
)

# Randomized Search
xgb_random = RandomizedSearchCV(
    estimator=xgb_clf_tuned,
    param_distributions=param_dist,
    n_iter=50,
    cv=3,
    verbose=2,
    random_state=42,
    n_jobs=-1
)

# Fit the model
xgb_random.fit(X_train, y_train)

# Best parameters
print("Best Parameters:", xgb_random.best_params_)

# Evaluate on test set
best_xgb = xgb_random.best_estimator_
y_pred_best_xgb = best_xgb.predict(X_test)
y_proba_best_xgb = best_xgb.predict_proba(X_test)[:, 1]

# evaluate metrics
print("Classification Report (Tuned XGBoost):")
print(classification_report(y_test, y_pred_best_xgb))
print("ROC-AUC:", roc_auc_score(y_test, y_proba_best_xgb))
```

Fitting 3 folds for each of 50 candidates, totalling 150 fits
Best Parameters: {'subsample': 0.8, 'scale_pos_weight': 3.5,

```
'n_estimators': np.int64(400), 'min_child_weight': 1, 'max_depth': 10,
'learning_rate': 0.2, 'gamma': 0.5, 'colsample_bytree': 0.6}
Classification Report (Tuned XGBoost):
```

	precision	recall	f1-score	support
0	0.95	0.92	0.93	7000
1	0.75	0.82	0.78	2000
accuracy			0.90	9000
macro avg	0.85	0.87	0.86	9000
weighted avg	0.90	0.90	0.90	9000

ROC-AUC: 0.9608351428571428

```
# Custom threshold evaluation
```

```
threshold = 0.35
```

```
y_pred_custom = (y_proba_best_xgb >= threshold).astype(int)
```

```
print("\nClassification Report (Threshold = 0.35):")
```

```
print(classification_report(y_test, y_pred_custom))
```

```
print("ROC-AUC:", roc_auc_score(y_test, y_proba_best_xgb))
```

```
Classification Report (Threshold = 0.35):
```

	precision	recall	f1-score	support
0	0.96	0.89	0.92	7000
1	0.69	0.87	0.77	2000
accuracy			0.88	9000
macro avg	0.83	0.88	0.85	9000
weighted avg	0.90	0.88	0.89	9000

ROC-AUC: 0.9608351428571428

```
# Save everything needed for production
```

```
joblib.dump(best_xgb, "models/final_classifier_xgboost.pkl")
```

```
joblib.dump(scaler, "models/classification_scaler.pkl")
```

```
['models/classification_scaler.pkl']
```

```
import joblib
```

```
# Load your trained XGBoost classifier
```

```
classifier = joblib.load("models/final_classifier_xgboost.pkl")
```

```
print(classifier.get_booster().feature_names)
```

```
['credit_score', 'credit_history_length', 'income', 'dti_ratio',
'default_history', 'home_ownership_OTHER', 'home_ownership_RENT',
'loan_purpose_EDUCATION', 'loan_purpose_HOMEIMPROVEMENT',
```

```
'loan_purpose_MEDICAL', 'loan_purpose_PERSONAL',  
'loan_purpose_VENTURE']
```

4.2 Regression Model For Loan AMount

The Regression analysis focuses on determining how much a client qualifies for a loan. The target variable is loan_amount. We will train and evaluate three models and all the models will use a test size of 20% and random state 42, the models include:

- Linear Regression
- Random Forest Regressor
- XGBoost Regressor

4.2.1 Linear Regression Model

```
# define X and y  
y = data_regresl_approved['loan_amount']  
X = data_regresl_approved.drop('loan_amount', axis=1)  
  
# Encode categorical columns for regression  
# One-hot encode with drop_first  
X_encoded = pd.get_dummies(X, columns=['home_ownership',  
    'loan_purpose'], drop_first=True)  
  
print(f"Original X shape: {X.shape}")  
print(f"Encoded X shape: {X_encoded.shape}")  
  
Original X shape: (10000, 7)  
Encoded X shape: (10000, 12)  
  
# train test split  
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y,  
    test_size=0.2, random_state=42)  
  
# Log transform features (income, dti_ratio) on BOTH train and test  
feature_log_cols = ['income', 'dti_ratio']  
for col in feature_log_cols:  
    X_train[col] = np.log1p(X_train[col])  
    X_test[col] = np.log1p(X_test[col])  
  
# Log transform target (loan_amount) on TRAIN ONLY  
y_train_log = np.log1p(y_train)  
  
# Apply IQR clipping  
def clip_iqr_columns(df, columns, iqr_multiplier=1.5):  
    """Clip outliers using IQR method"""  
    df_clipped = df.copy()  
  
    for col in columns:  
        if col in df.columns:  
            Q1 = df[col].quantile(0.25)
```

```

Q3 = df[col].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - iqr_multiplier * IQR
upper_bound = Q3 + iqr_multiplier * IQR

# Count outliers before clipping
outliers_low = (df[col] < lower_bound).sum()
outliers_high = (df[col] > upper_bound).sum()
total_outliers = outliers_low + outliers_high

if total_outliers > 0:
    print(f" {col}: Clipping {total_outliers} outliers")
    df_clipped[col] = df[col].clip(lower_bound,
upper_bound)

    return df_clipped

# Apply to numerical columns (after log transform, before scaling)
numerical_cols = ['credit_score', 'income', 'dti_ratio',
'credit_history_length']

# Apply StandardScaler to numerical features
scale_cols = ['credit_score', 'income', 'dti_ratio',
'credit_history_length']

# Initialize the scaler
scaler = StandardScaler()

# Fit on TRAIN data only, transform both train and test
X_train[scale_cols] = scaler.fit_transform(X_train[scale_cols])
X_test[scale_cols] = scaler.transform(X_test[scale_cols])

# check th distribution
print(f"Traning data shape: {X_train.shape}")
print(f"Testing data shape: {X_test.shape}")

Traning data shape: (8000, 12)
Testing data shape: (2000, 12)

```

4.2.1 Linear Regression Model

```

# Intialize and fit
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train_log)

LinearRegression()

# predict on test data
y_pred_lin = lin_reg.predict(X_test)
y_pred_lin

```

```

array([10.232134 ,  9.16651054,  8.73086751, ...,  8.40597864,
        7.93336939,  9.72111578], shape=(2000,))

# Convert back to original scale
y_pred_original = np.expml(y_pred_lin)
y_pred_original

array([27780.73311795,  9570.16814607,  6190.09659866, ...,
        4472.73379666,  2787.80757986, 16664.82973165], shape=(2000,))

#evaluation
rmse = np.sqrt(mean_squared_error(y_test, y_pred_original))
mae = mean_absolute_error(y_test, y_pred_original)
r2 = r2_score(y_test, y_pred_original)

print("\nLinear Regression Results (Original Units):")
print("RMSE:", rmse)
print("MAE:", mae)
print("R²:", r2)

Linear Regression Results (Original Units):
RMSE: 2427.436722701981
MAE: 1473.1205160277432
R²: 0.8873291279247202

```

4.2.2 Random Forest Regression Model

```

# intialiaxe and fit
rf_reg = RandomForestRegressor()
rf_reg.fit(X_train, y_train_log)

RandomForestRegressor()

# predict
y_pred_rf = rf_reg.predict(X_test)

# Convert back to original scale
y_pred_original = np.expml(y_pred_rf)
y_pred_original

array([31326.2991427 , 11018.22269148,  6044.16384525, ...,
        4999.37937869,  3218.71209974, 15215.87781394], shape=(2000,))

#evaluation
rmse = np.sqrt(mean_squared_error(y_test, y_pred_original))
mae = mean_absolute_error(y_test, y_pred_original)
r2 = r2_score(y_test, y_pred_original)

print("\nRandom Forest Regression Results (Original Units):")
print("RMSE:", rmse)

```



```
print("MAE:", mae)
print("R²:", r2)
```

Random Forest Regression Results (Original Units):

RMSE: 419.2784223031336

MAE: 168.1242740796289

R²: 0.9966385949724685

Random Forest Regression Hypeparameter tuning

```
# Define the parameter grid
param_dist = {
    'n_estimators': np.arange(100, 500, 50),
    'max_depth': [3, 5, 7, 10],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['auto', 'sqrt', 'log2'],
    'bootstrap': [True, False]
}

# Initialize the model
rf_reg2 = RandomForestRegressor(random_state=42)

# Randomized Search
random_search = RandomizedSearchCV(
    estimator=rf_reg2,
    param_distributions=param_dist,
    n_iter=50,
    cv=3,
    verbose=2,
    random_state=42,
    n_jobs=-1
)

# Fit on training data
random_search.fit(X_train, y_train_log)

# Best parameters
print("Best Parameters:", random_search.best_params_)

# Best model
best_rf_la = random_search.best_estimator_

# Predict and evaluate
y_pred_rf = best_rf_la.predict(X_test)
y_pred_original = np.expml(y_pred_rf)

rmse = np.sqrt(mean_squared_error(y_test, y_pred_original))
mae = mean_absolute_error(y_test, y_pred_original)
```

```

r2 = r2_score(y_test, y_pred_original)

print("\nTuned Random Forest Results (Original Units):")
print("RMSE:", rmse)
print("MAE:", mae)
print("R²:", r2)

Fitting 3 folds for each of 50 candidates, totalling 150 fits
Best Parameters: {'n_estimators': np.int64(450), 'min_samples_split':
2, 'min_samples_leaf': 2, 'max_features': 'sqrt', 'max_depth': 10,
'bootstrap': False}

Tuned Random Forest Results (Original Units):
RMSE: 2682.4132224380137
MAE: 1441.5938266105604
R²: 0.8624162372539685

```

4.2.3 XGBoost Regression Model

```

# initialize and fit
xgb_reg = XGBRegressor()
xgb_reg.fit(X_train, y_train_log)

XGBRegressor(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None,
              early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None,
              feature_types=None,
              feature_weights=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=None, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None,
              max_depth=None,
              max_leaves=None, min_child_weight=None, missing=nan,
              monotone_constraints=None, multi_strategy=None,
              n_estimators=None,
              n_jobs=None, num_parallel_tree=None, ...)

# predict
y_pred_xgb = xgb_reg.predict(X_test)

# Convert back to original scale
y_pred_original = np.expml(y_pred_xgb)
y_pred_original

array([33925.914 , 10557.048 , 5516.7085, ..., 4992.103 ,
       3101.4917,
       15213.234 ], shape=(2000,), dtype=float32)

```

```

# evaluation
rmse = np.sqrt(mean_squared_error(y_test, y_pred_original))
mae = mean_absolute_error(y_test, y_pred_original)
r2 = r2_score(y_test, y_pred_original)

print("\nXGBoost Regression Results (Original Units):")
print("RMSE:", rmse)
print("MAE:", mae)
print("R²:", r2)

```

```

XGBoost Regression Results (Original Units):
RMSE: 465.04686563182946
MAE: 229.26095440673828
R²: 0.9958646786506812

```

XGBoost Classifier Hyperparameter tuning

```

# Define the parameter grid
param_dist = {
    'n_estimators': np.arange(100, 500, 50),
    'max_depth': [3, 5, 7, 10],
    'learning_rate': [0.01, 0.05, 0.1, 0.2],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
    'gamma': [0, 0.1, 0.2, 0.5],
    'reg_alpha': [0, 0.01, 0.1, 1],
    'reg_lambda': [1, 1.5, 2]
}

# Initialize the model
xgb_reg_tuned = XGBRegressor(random_state=42,
                             objective='reg:squarederror')

# Randomized Search
random_search_xgb = RandomizedSearchCV(
    estimator=xgb_reg_tuned,
    param_distributions=param_dist,
    n_iter=50,
    cv=3,
    verbose=2,
    random_state=42,
    n_jobs=-1
)

# Fit on training data
random_search_xgb.fit(X_train, y_train_log)

# Best parameters
print("Best Parameters:", random_search_xgb.best_params_)

```

```

# Best model
best_xgb = random_search_xgb.best_estimator_

# Predict and evaluate
y_pred_xgb = best_xgb.predict(X_test)
y_pred_original = np.expml(y_pred_xgb)

rmse = np.sqrt(mean_squared_error(y_test, y_pred_original))
mae = mean_absolute_error(y_test, y_pred_original)
r2 = r2_score(y_test, y_pred_original)

print("\nTuned XGBoost Results (Original Units):")
print("RMSE:", rmse)
print("MAE:", mae)
print("R²:", r2)

Fitting 3 folds for each of 50 candidates, totalling 150 fits
Best Parameters: {'subsample': 1.0, 'reg_lambda': 1.5, 'reg_alpha':
0.01, 'n_estimators': np.int64(350), 'max_depth': 7, 'learning_rate':
0.2, 'gamma': 0, 'colsample_bytree': 1.0}

Tuned XGBoost Results (Original Units):
RMSE: 486.2880923752457
MAE: 200.29752578735352
R²: 0.9954782859879304

# Save the best model (Random Forest Baseline)
joblib.dump(rf_reg, "models/best_loan_amount_predictor.pkl")
joblib.dump(scaler, "models/regression_scaler.pkl")

print("✅ Best regression model saved!")

✅ Best regression model saved!

```

4.3 Interest Segmentation For Interest Rate

This Regression analysis focuses on determining interest rate charged for a given a loan under a risk-based pricing framework. The target variable is interest rate. We will train and evaluate three models and all the models will use a test size of 20% and random state 42, the models include:

- Linear Regression
- Random Forest Regressor
- XGBoost Regressor

```

# Encode home_ownership, loan_purpose
data_int_encoded = pd.get_dummies(data_int, columns=['home_ownership',
'loan_purpose'], drop_first=True)

```

```

# Defirn X and y
X = data_int_encoded.drop('interest_rate', axis=1)
y = data_int_encoded['interest_rate']

# scale the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# fin K
inertias = []
sil_scores = []
K_range = range(2, 10)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    inertias.append(kmeans.inertia_)
    sil_scores.append(silhouette_score(X_scaled, kmeans.labels_))

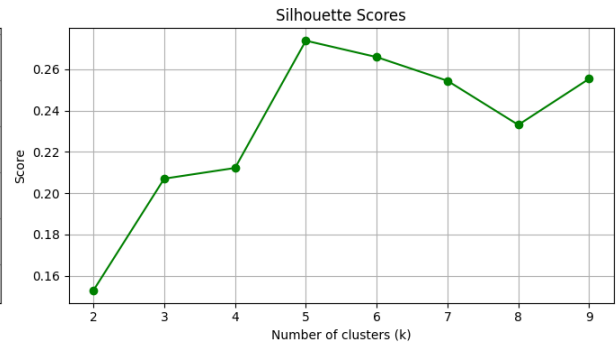
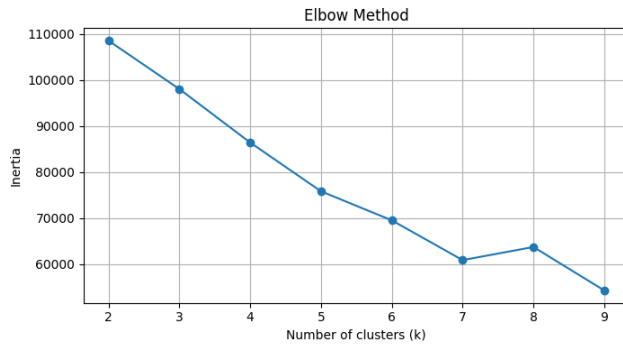
# Plot elbow method
plt.figure(figsize=(14,4))

plt.subplot(1, 2, 1)
plt.plot(K_range, inertias, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Inertia")
plt.xticks(K_range)
plt.grid(True)

# Plot silhouette scores
plt.subplot(1, 2, 2)
plt.plot(K_range, sil_scores, marker='o', color="green")
plt.title("Silhouette Scores")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Score")
plt.xticks(K_range)
plt.grid(True)

plt.tight_layout()
plt.show()

```



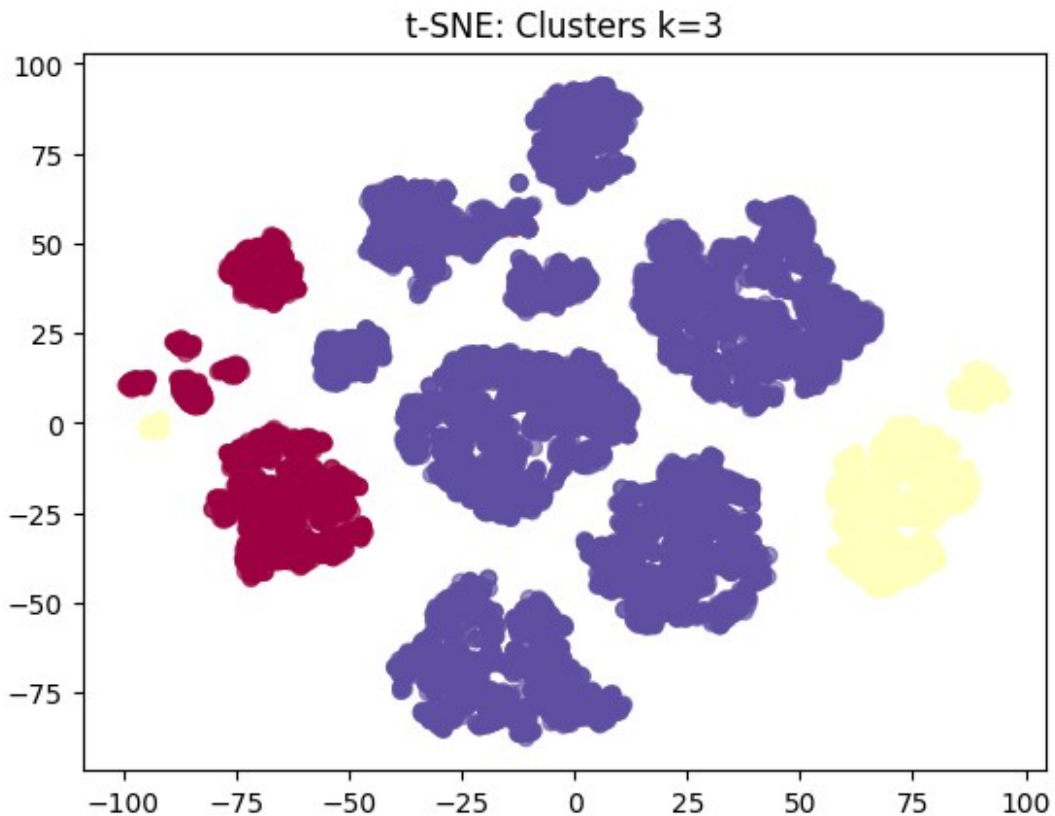
The elbow method analysis indicates potential optimal cluster counts at k=4 and k=5, as both represent points of diminishing returns in inertia reduction. To resolve this ambiguity and select the most appropriate number of clusters, I conduct further validation using silhouette score analysis. The scores showed that K=5 is better than 4

```
# view cluster labels
kmeans = KMeans(n_clusters=3, random_state=42, max_iter=1000)
cluster_labels = kmeans.fit_predict(X_scaled)

# preview
cluster_labels[:10]

array([2, 2, 2, 2, 1, 2, 2, 2, 1, 1], dtype=int32)

# visualize the cluster using tsne
tsne = TSNE(n_components=2, random_state=42, perplexity=30)
X_tsne = tsne.fit_transform(X_scaled)
plt.scatter(X_tsne[:,0], X_tsne[:,1], c=cluster_labels,
            cmap='Spectral', alpha=0.6)
plt.title('t-SNE: Clusters k=3')
plt.show()
```



```
# Distances in the k-means feature space
centroids = kmeans.cluster_centers_
assigned = centroids[cluster_labels]
distances = np.linalg.norm(X_scaled - assigned, axis=1)
```

```
# Normalize and scale to premium adjustment
premium_adjustment = (distances / distances.max()) * 10
```

```
# Attach to the original approved-loans frame
data_int['premium_adjustment'] = premium_adjustment
```

```
# Adding the cluster_labels to the data int
data_int['cluster'] = cluster_labels
data_int_encoded["cluster"] = cluster_labels
```

```
# preview
data_int.iloc[1010:1015]
```

	credit_score	credit_history_length	income \
2508	706	2.00	22402.00
2510	630	2.00	22805.00
2512	495	4.00	23480.00
2514	670	2.00	23799.00
2515	530	2.00	24037.00

	dti_ratio	default_history	loan_amount \
2508	0.13	0	3000.00
2510	0.13	0	3000.00
2512	0.13	0	3000.00
2514	0.13	0	3000.00
2515	0.12	0	3000.00

	home_ownership	loan_purpose	interest_rate \
2508	RENT	PERSONAL	11.86
2510	RENT	MEDICAL	9.70
2512	RENT	HOMEIMPROVEMENT	10.99
2514	RENT	VENTURE	13.06
2515	RENT	VENTURE	12.99

	premium_adjustment	cluster
2508	1.95	2
2510	1.50	2
2512	1.89	0
2514	1.17	1
2515	1.62	1

```
# Analyze the real risk factors for each cluster
cluster_stats = data_int.groupby('cluster').agg({
    'credit_score': 'mean',
    'interest_rate': 'mean',
    'default_history': 'mean',
    'dti_ratio': 'mean',
    'income': 'mean'
}).round(2)
```

```
# Sort by risk
cluster_stats['risk_score'] = (
    (cluster_stats['credit_score'].max() -
    cluster_stats['credit_score']) +
    (cluster_stats['interest_rate'] -
    cluster_stats['interest_rate'].min())
)
```

```
cluster_stats['risk_rank'] = cluster_stats['risk_score'].rank()
cluster_stats[['credit_score', 'interest_rate', 'default_history',
'risk_score', 'risk_rank']].sort_values('risk_rank')
```

\ cluster	credit_score	interest_rate	default_history
1	633.50	12.89	0.00
2	632.03	12.87	0.00

0	629.89	12.77	0.00
---	--------	-------	------

	risk_score	risk_rank
cluster		
1	0.12	1.00
2	1.57	2.00
0	3.61	3.00

Rename the clusters

```
cluster_names = {
    1: "Prime",
    2: "Preferred",
    0: "Standard"
}
```

```
data_int['cluster_name'] = data_int['cluster'].map(cluster_names)
data_int[['cluster', 'cluster_name']].head()
```

	cluster	cluster_name
0	2	Preferred
2	2	Preferred
3	2	Preferred
4	2	Preferred
5	1	Prime

Assign Base Premiums per Risk Band

```
premium_mapping = {
    "Prime": 0.3,
    "Preferred": 0.6,
    "Standard": 1.0
}
```

```
data_int['base_premium'] =
data_int['cluster_name'].map(premium_mapping)
data_int[['cluster_name', 'base_premium']].head()
```

	cluster_name	base_premium
0	Preferred	0.60
2	Preferred	0.60
3	Preferred	0.60
4	Preferred	0.60
5	Prime	0.30

create the final rate

```
margin = 0.5
data_int['final_rate'] = data_int['base_premium'] +
data_int['premium_adjustment'] + margin
```

Preview

```
data_int[['cluster_name', 'base_premium', 'premium_adjustment', 'final_rate']].head()
```

	cluster_name	base_premium	premium_adjustment
0	Preferred	0.60	2.99
2	Preferred	0.60	2.12
3	Preferred	0.60	2.66
4	Preferred	0.60	2.88
5	Prime	0.30	3.96

```
# final kesonia fomula
```

```
data_int['kesonic_rate_pct'] = 10.09 + data_int['final_rate']
```

```
print("Final Kesonic Rates (in %):")
```

```
data_int[['cluster_name', 'base_premium', 'premium_adjustment', 'final_rate', 'kesonic_rate_pct', 'interest_rate']].head(10)
```

```
Final Kesonic Rates (in %):
```

	cluster_name	base_premium	premium_adjustment	\
0	Preferred	0.60	2.99	
2	Preferred	0.60	2.12	
3	Preferred	0.60	2.66	
4	Preferred	0.60	2.88	
5	Prime	0.30	3.96	
6	Preferred	0.60	2.72	
7	Preferred	0.60	2.49	
8	Preferred	0.60	2.81	
9	Prime	0.30	3.82	
10	Prime	0.30	2.12	

	final_rate	kesonic_rate_pct	interest_rate
0	4.09	14.18	16.02
2	3.22	13.31	12.87
3	3.76	13.85	15.23
4	3.98	14.07	14.27
5	4.76	14.85	7.14
6	3.82	13.91	12.42
7	3.59	13.68	11.11
8	3.91	14.00	8.90
9	4.62	14.71	14.74
10	2.92	13.01	10.37

```
# 1. Save K-means model
joblib.dump(kmeans, 'models/kmeans_model.pkl')

# 2. Save scaler
joblib.dump(scaler, 'models/kesonic_scaler.pkl')

# 3. Save max distance for premium calculation
np.save('models/max_distance.npy', distances.max())

# 4. Save cluster names and premium mapping
cluster_info = {
    'cluster_names': cluster_names,
    'premium_mapping': premium_mapping,
    'feature_names': list(X.columns),
    'max_distance': distances.max(),
    'kesonia_base': 10.09,
    'margin': 0.5
}

with open('models/cluster_info.pkl', 'wb') as f:
    pickle.dump(cluster_info, f)
```